

Evaluations Using LangSmith

Overview

In this exercise you will evaluate an LLM-based intent classifier for an e-commerce customer support system. The classifier predicts which category a customer ticket belongs to — `order_status`, `refund_request`, `product_issue`, `account_help`, or `other`.

You will run automated evaluations using Exact Match, manually annotate failures, define failure categories from your observations, make one focused prompt improvement, and compare the before-and-after results. This is an optional LangSmith UI companion to the main Customer Support Agent Evaluation handout; it does not replace the required spreadsheet or notebook deliverables.

Exercise at a glance

Dataset: 100 e-commerce customer support tickets

Categories: `order_status` · `refund_request` · `product_issue` · `account_help` · `other`

Evaluators: Exact Match; optional LLM-as-a-Judge

Phase	What you do	Tool
1. Dataset	Upload CSV with correct column mapping	LangSmith UI
2. Playground	Configure classifier prompt and output schema	LangSmith Playground
3. Exact Match	Run eval on all 100 examples, find failures	LangSmith Playground
4. Annotation	Read failures, define categories, assign each	Annotation Queue
5. Improve + compare	Make one focused prompt change, rerun, and check regressions	LangSmith Playground

1. Upload Your Dataset

Your CSV needs exactly three columns. The column names matter — they become the keys that evaluators read. Remove any other columns before uploading.

Column	Maps to	Example value
<code>query</code>	Input — the customer ticket	Hi, my order hasn't arrived...
<code>intent</code>	Output — ground truth label	<code>order_status</code>
<code>id</code>	Metadata — for tracking	t000

Important

Remove all other columns before uploading — `predicted_category`, `reasoning`, `correct`, etc. Those are eval results, not dataset inputs.

Steps

1. Left sidebar → Datasets & Experiments → + New Dataset → name it → upload your CSV.

2. Review the column mapping and correct it if needed. Set: query → Input · intent → Output · id → Metadata.
3. Check the preview panel. It must show the query as input, intent as output, id as metadata. If correct → click Create.

2. Configure the Playground

Set up the classifier prompt, output schema, and model. Do this once before running any experiment.

Classifier prompt

Paste this as the System message:

```
You are a triage system for an e-commerce support inbox.
Classify the customer's ticket into EXACTLY ONE of these categories:
- order_status: questions about where an order is, tracking, delivery ETA
- refund_request: the customer wants their money back
- product_issue: the item arrived broken, wrong, defective, or not as described
- account_help: login, password, address, payment method changes
- other: anything that doesn't fit the above (general questions, feedback, browsing)
Return only the intent key.
```

Human message

Add a Human message with the variable {query}:

```
{query}
```

Format toggle: f-string · Click the {query} chip → map to input.query.

Output schema

Click Output schema → + Property → name: intent · type: String · Required: on. This forces the model to return a clean structured output like {"intent": "order_status"} which evaluators can reliably read.

Model

Model: gpt-4o-mini · Temperature: 0 · Set up Evaluation → select your dataset.

3. Run Exact Match Evaluation

Exact Match is a free, code-based evaluator. It compares output.intent against referenceOutput.intent and returns 1 (correct) or 0 (wrong). No tokens are consumed. Run this first — no LLM judge yet.

Steps

1. Click + Evaluator → Quality templates → Exact Match. Nothing else at this stage.
2. Select all dataset rows → Start. The average intent_exact_match score at the top of the column is your baseline accuracy. Write it down.
3. Filter intent_exact_match = 0. These are your failures. Select all filtered rows → Annotate → choose your annotation queue.

4. Annotate Failures

This is where the human analysis happens. The key rule: read every failure and write notes before you define any categories. Your taxonomy must emerge from what you actually observe — not assumptions made in advance.

Key rule

Read ALL failures and write a note on each one BEFORE you try to define any categories. Categories defined before reading the data will miss the real patterns.

Part 1 — Read all failures

Open Annotation Queues → your queue → Go To Queue. For each run: read the query, what the model predicted, and what the correct answer was. Write a short comment explaining why the model got confused. Do NOT categorise yet.

Part 2 — Define failure categories

After reading every failure, look for patterns across your notes. Define categories that describe the mechanism of failure — not just 'wrong answer.'

Part 3 — Add rubric and assign

Queue settings → + Add feedback rubric → Key: failure_category · Type: Categorical · Options: your discovered categories · Required: on. Then go back through the queue and assign a category to every failure. Click Done after each.

5. Improve, Compare, and Optionally Add an LLM Judge

Before adding an LLM judge, choose the highest-impact failure category, make one focused classifier-prompt change, and rerun all dataset rows under a new experiment name. Use LangSmith Comparison to record the accuracy delta, inspect wins and regressions, and confirm that the change did not damage previously passing cases. The LLM judge below is optional and should be attempted only after the human analysis and before-and-after comparison are complete.

The judge sees the query and the predicted category. It does not see the ground truth. It uses its own semantic understanding to decide: given this query, was this prediction correct? One or zero. We run it category by category to tune it precisely for each failure type.

What the judge does and does not see

Sees: the customer query + the classifier's predicted category

Does NOT see: the ground truth label

Output: 1 (prediction was correct) or 0 (prediction was wrong) + one-sentence reasoning

Purpose: assist with scalable review after calibration; continue periodic human checks

Setup steps

1. In the Playground → + Evaluator → LLM-as-a-Judge → Create from scratch. Do NOT create from the Evaluators sidebar — create from inside the Playground.
2. Set the first message type to SYSTEM. Paste:

```
You are evaluating an e-commerce support intent classifier.
The classifier predicts one of these categories:
order_status, refund_request, product_issue, account_help, other.
Given the query and the predicted category, decide if the prediction is correct.
Return JSON only: {"score": 0|1, "reasoning": "one sentence"}
```

3. Click + Message → set to HUMAN. Paste:

```
Query: {{query}}
Predicted category: {{output}}
```



4. Map variables: click {{query}} chip → select input.query (turns blue). Click {{output}} chip → select output.intent (stays yellow). No ground truth variable.

Feedback configuration

Setting	Value
Feedback key	intent_llm_judge
Description	Given the customer query and the classifier's predicted intent, determine whether the prediction is correct. Score 1 if correct, 0 if incorrect.
Response format	Score
Min / Max	0 / 1
Include reasoning	On

Running category by category

Save the evaluator. Do NOT click Test — run directly from the experiment.

Filter by one failure_category value you discovered → select those rows → Start. Then check the judge scores against your human annotations:

Judge score	On a failure row	Action
0	Aligned — judge agrees the prediction was wrong	Move to next category
1	Check: was the ground truth label wrong? Or is the judge miscalibrated?	Investigate, refine prompt if needed

Goal

Once the judge is aligned with the current human-labelled calibration set, it can assist with reviewing future outputs at scale. Continue sampling human-reviewed cases, and recalibrate after changes to the classifier prompt, model, categories, or data distribution.

What to submit

Use the main Customer Support Agent Evaluation handout as the authoritative checklist. The summary below shows how this optional LangSmith companion supports that submission.

- Completed spreadsheet: reviewed/corrected ground truth labels, model predicted labels, PASS/FAIL comparison, and participant-assigned failure categories.
- Failure analysis: brief category names, grouped examples, and the most important failure category to improve.
- Prompt/router fix: screenshot or summary of the change, plus which failure category the fix targets.
- Regression test results: failed examples re-run after the fix plus at least one previously passing example.
- LangSmith evidence: dataset or experiment link, or screenshots showing the baseline run, failed cases, and improved run. If you complete the optional LLM-as-a-Judge extension, also include judge results, a human-vs-judge comparison, and any judge model or prompt changes.

- Short Loom walkthrough: what changed, what improved, what still fails, and what you would try next.